

Training a New Tokenizer from an Old Tokenizer

If there is no language model for the language you want, or if your data is very different from the data used to train existing models, then you may need to create a new tokenizer.

Simply put:

If you want to train a model from scratch, you usually need to train a tokenizer first.

This is because the tokenizer determines how text will be split into small tokens or subwords.

What is tokenizer training?

Transformer models usually do not process text as full words. Instead, they break text into smaller units called **subwords**.

The tokenizer's job is to determine:

- Which subwords appear most frequently
- Which subwords are most useful for the dataset

To do this, the tokenizer needs to analyze all the text in the corpus.

This learning process is called **tokenizer training**.

A very important point

Tokenizer training and model training are **not the same thing**.

Model training

During model training:

- **Stochastic Gradient Descent (SGD)** is used
- The objective is to **minimize loss**
- The process contains randomness

Because of this, running the same training twice can produce slightly different results.

If you want identical results, you usually need to **set a random seed**.

Tokenizer training

During tokenizer training:

- The text is **analyzed statistically**
- The algorithm decides **which subwords should be kept**

This is a **statistical process** and it is **deterministic**.

That means:

If you use the same algorithm and the same corpus, the result will always be identical.

Building the Corpus

The **Transformers** library provides a simple API:

```
AutoTokenizer.train_new_from_iterator()
```

This allows you to train a new tokenizer while keeping the characteristics of an existing tokenizer.

Suppose we want to retrain **GPT-2**, but instead of general English text we use a different type of data.

In this example, instead of natural language we will use **Python source code** as the language.

Loading the Dataset

To collect Python code data, the **datasets** library is used.

The **CodeSearchNet dataset** contains many functions from open-source GitHub projects.

Here we load the Python portion:

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset("code_search_net", "python")
```

This may take some time to load.

What is inside the training split?

```
raw_datasets["train"]
```

Output:

```
Dataset({
  features: ['repository_name', 'func_path_in_repository', 'func_name', 'whole_func_string',
'language',
  'func_code_string', 'func_code_tokens', 'func_documentation_string',
'func_documentation_tokens', 'split_name',
  'func_code_url'
],
  num_rows: 412178
})
```

This shows that the dataset contains many columns.

For tokenizer training we will use:

`whole_func_string`

because it contains the **entire function as a single string**.

Looking at one example

```
print(raw_datasets["train"][123456]["whole_func_string"])
```

This will print a Python function.

The idea here is:

We want to **teach the tokenizer how Python code looks**.

The main task

We must provide the dataset to the tokenizer in a way that:

- Data is passed in **small batches**
- All data is **not loaded into RAM at once**
- Memory does not get exhausted

For this purpose, we use an **iterator or generator**.

Why a list of texts is needed

If the tokenizer receives text in **batches**, it works faster.

So we provide texts in batches.

For example:

Each batch contains **1000 texts**.

What is not a good approach

The following code loads **all data into memory**:

```
training_corpus = [raw_datasets["train"][i: i + 1000]["whole_func_string"] for i in range(0, len(raw_datasets["train"]), 1000)]
```

This is fine for small datasets.

But for large datasets it may cause **RAM issues**.

Explanation of the Code

```
training_corpus = [raw_datasets["train"][i: i + 1000]["whole_func_string"] for i in range(0, len(raw_datasets["train"]), 1000)]
```

1) What this line does in simple terms

This line splits the dataset `raw_datasets["train"]` into chunks of **1000 rows**, then extracts the column `"whole_func_string"` from each chunk and stores them in a list called `training_corpus`.

Meaning:

- The entire training dataset is used
- It is divided into chunks of size **1000**
- From each chunk only the `"whole_func_string"` field is taken
- All chunks are stored together in a list

2) Breaking the line into parts

There are three main parts.

A. `range(0, len(raw_datasets["train"]), 1000)`

This generates index values.

Suppose the dataset has **4500 rows**.

Then this will produce:

0, 1000, 2000, 3000, 4000

Meaning:

- First chunk starts at index **0**
- Second chunk starts at **1000**
- Third chunk starts at **2000**
- And so on

Where:

- `0` = starting index
- `len(raw_datasets["train"])` = end of dataset
- `1000` = step size

B. `raw_datasets["train"][i: i + 1000]`

This slices **1000 rows** from the dataset.

If `i = 0`:

```
raw_datasets["train"][0:1000]
```

This means the **first 1000 rows**.

If `i = 1000`:

```
raw_datasets["train"][1000:2000]
```

This means the **next 1000 rows**.

If `i = 4000`:

```
raw_datasets["train"][4000:5000]
```

But if the dataset only has **4500 rows**, then it will take rows **4000–4499**.

C. `["whole_func_string"]`

This extracts the `"whole_func_string"` column from the sliced chunk.

For example, if a chunk contains:

```
[
  {"whole_func_string": "def add(a,b): return a+b", "label": 1},
  {"whole_func_string": "def sub(a,b): return a-b", "label": 0}
]
```

Then selecting `"whole_func_string"` gives:

```
[
  "def add(a,b): return a+b",
  "def sub(a,b): return a-b"
]
```

3) The same logic written with a loop

The one-line list comprehension is equivalent to:

```
training_corpus = []

for i in range(0, len(raw_datasets["train"]), 1000):
    chunk = raw_datasets["train"][i:i+1000]
    func_strings = chunk["whole_func_string"]
    training_corpus.append(func_strings)
```

Steps:

- Create an empty list
- Move through the dataset in steps of **1000**
- Slice **1000 rows at a time**
- Extract `"whole_func_string"`
- Append it to the list

4) What `training_corpus` looks like

`training_corpus` becomes a **nested list**.

Example:

```
training_corpus = [  
    ["func1", "func2", ..., "func1000"],  
    ["func1001", "func1002", ..., "func2000"],  
    ["func2001", ..., "func3000"]  
]
```

So:

- `training_corpus[0]` → first 1000 functions
- `training_corpus[1]` → next 1000 functions

Better Approach: Using a Generator

```
training_corpus = (  
    raw_datasets["train"][i : i + 1000]["whole_func_string"]  
    for i in range(0, len(raw_datasets["train"]), 1000)  
)
```

What happens here:

- Data is **not loaded immediately**
- Only a **generator object** is created
- When needed, **1000 texts are loaded**
- Then the next 1000 are loaded

This saves memory.

Generator Limitation

A generator can only be used **once**.

Example:

```
gen = (i for i in range(10))  
print(list(gen))  
print(list(gen))
```

Output:

```
[0,1,2,3,4,5,6,7,8,9]
[]
```

The second time returns nothing because the generator is **exhausted**.

Solution: Wrap it in a function

```
def get_training_corpus():
    return (
        raw_datasets["train"][i : i + 1000]["whole_func_string"]
        for i in range(0, len(raw_datasets["train"]), 1000)
    )
```

```
training_corpus = get_training_corpus()
```

Every time `get_training_corpus()` is called, a **new generator** is created.

Another clean method: using **yield**

```
def get_training_corpus():
    dataset = raw_datasets["train"]
    for start_idx in range(0, len(dataset), 1000):
        samples = dataset[start_idx : start_idx + 1000]
        yield samples["whole_func_string"]
```

Explanation:

- `dataset` = training data
- `start_idx` increases as `0, 1000, 2000...`
- Each step takes **1000 samples**
- `"whole_func_string"` is returned

Because `yield` is used, the function behaves as a **generator**.

Training a New Tokenizer

Now our corpus is ready as an **iterator in batch form**, so we are prepared to train a new tokenizer.

The first step is to load the **old tokenizer of the model** that we want to use. Here, **GPT-2** is used as an example.

```
from transformers import AutoTokenizer
```

```
old_tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Even though we will train a new tokenizer, loading the old tokenizer is a good idea.

Because this way we **do not have to start from zero**.

Advantages:

- We do not need to specify the tokenization algorithm again
- We do not need to redefine special tokens
- The structure of the new tokenizer will remain the same as GPT-2

Only the **vocabulary will change**.

And that vocabulary will be determined by our corpus.

So the tokenizer system remains the same, but the **vocabulary adapts to the new dataset**.

How the Old Tokenizer Splits an Example

Suppose we have this function:

```
example = """def add_numbers(a, b):  
    """Add the two numbers `a` and `b`."""  
    return a + b"""
```

Tokenizing it:

```
tokens = old_tokenizer.tokenize(example)  
tokens
```

Output:

```
['def', 'Ġadd', 'Ġ_', 'Ġn', 'Ġumbers', 'Ġ(', 'Ġa', 'Ġ,', 'ĠĠb', 'Ġ):', 'ĠĠ', 'ĠĠ', 'ĠĠ', 'ĠĠ', 'ĠĠ', 'ĠAdd', 'Ġthe', 'Ġtwo', 'Ġnumbers', 'ĠĠ', 'Ġa', 'Ġ,', 'ĠĠand', 'ĠĠ', 'Ġb', 'Ġ,', 'Ġ.', 'Ġ', 'Ġ', 'Ġ', 'ĠĠ', 'ĠĠ', 'ĠĠ', 'ĠĠ', 'Ġreturn', 'ĠĠa', 'ĠĠ+', 'ĠĠb']
```

Some special symbols appear:

- **Ġ** means a space
- **Ġ** means a newline

What is the problem here?

This tokenizer is **not very efficient for code**.

Reasons:

- Every space becomes a separate token
- Indentation is broken into multiple tokens
- In code, **4 or 8 spaces are common**, so treating them as one token would be better
- The function name `add_numbers` is split in a strange way
- The tokenizer is not very familiar with `_`

So a tokenizer designed for **plain English** does not handle code naturally.

Training a New Tokenizer

Now we train a new tokenizer:

```
tokenizer = old_tokenizer.train_new_from_iterator(training_corpus, 52000)
```

Here:

- `training_corpus` = our dataset
- `52000` = vocabulary size

So after training, the tokenizer will build a vocabulary of about **52,000 tokens**.

Is this training slow or fast?

If the corpus is large, it may take some time.

However, with a **fast tokenizer**, the process is surprisingly fast.

Important Note

`AutoTokenizer.train_new_from_iterator()` only works when the tokenizer is a **fast tokenizer**.

What is a Fast Tokenizer?

In the Transformers library there are two types of tokenizers.

1. Pure Python tokenizer

These are implemented entirely in Python.

2. Fast tokenizer

These are built on the **Tokenizers library**, which is written in **Rust**.

Why is a fast tokenizer necessary?

Python is very popular for data science.

But for **high-speed parallel processing**, Python alone is not always sufficient.

Where speed is critical, other languages are used.

Examples:

- Model matrix multiplication uses **CUDA / C libraries**
- Tokenizer training uses **Rust** for speed

What if tokenizer training were done in pure Python?

It would be **extremely slow**.

That is why the **Tokenizers library** was created.

Do you need to know Rust?

No.

Just like you do not need to learn CUDA to run models on GPUs, you do not need to learn Rust to use fast tokenizers.

Because the library provides **Python bindings**.

So you work in Python, but internally **Rust performs the fast computation**.

What operations become faster?

Fast tokenizers improve speed in:

- Training a new tokenizer
- Batch tokenization
- Parallel processing

What does AutoTokenizer do?

AutoTokenizer usually **automatically selects the fast tokenizer** if one is available.

So most of the time you do not need to manually think about it.

Testing the New Tokenizer

Now we tokenize the same example again using the new tokenizer.

```
tokens = tokenizer.tokenize(example)
tokens
```

Output:

```
['def', 'Ġadd', 'Ġ_', 'numbers', 'Ġ(', 'Ġa', 'Ġ,', 'ĠĠb', 'Ġ):', 'ĠĠĠĠĠ', 'ĠĠ''''', 'Add', 'Ġthe', 'Ġtwo', 'Ġnumbers', 'ĠĠ', 'a', 'Ġ,', 'Ġand', 'ĠĠ', 'b', 'Ġ,', 'Ġ.'''''', 'ĠĠĠĠĠ', 'Ġreturn', 'Ġa', 'Ġ+', 'Ġb']
```

What improved?

The symbols **Ġ** and **Ċ** are still present.

But now the tokenizer has learned **some patterns of Python code**.

Examples:

- **ĊĠĠĠĠ** = newline + indentation
- **Ġ''''** = beginning of a docstring

These patterns are very common in Python code.

So the tokenizer now recognizes them in a **more intelligent way**.

Better Handling of Function Names

Previously `add_numbers` was split awkwardly.

Now the tokenizer handles the function name **more naturally**.

This means it has learned **code-specific structure**.

The New Tokenizer is More Compact

The same code now uses **fewer tokens**.

```
print(len(tokens))  
print(len(old_tokenizer.tokenize(example)))
```

Output:

```
27  
36
```

So:

- New tokenizer → **27 tokens**
- Old tokenizer → **36 tokens**

This means the new tokenizer represents the same text with **fewer tokens**.

That is a sign of a good tokenizer.

Benefits:

- More compact representation
- Fewer unnecessary splits
- The model can understand code structure better

Another Example

```
example = """class LinearLayer():  
    def __init__(self, input_size, output_size):
```

```

self.weight = torch.randn(input_size, output_size)
self.bias = torch.zeros(output_size)

def __call__(self, x):
    return x @ self.weights + self.bias
"""

```

Tokenizing:

```
tokenizer.tokenize(example)
```

Output:

```

['class', 'ĠLinear', 'Layer', '():', 'ĠĠĠĠ', 'Ġdef', 'Ġ__', 'init', '__(', 'self', ',', 'Ġinput', '_, 'size', ',',
'Ġoutput', '_, 'size', '):', 'ĠĠĠĠĠĠĠĠ', 'Ġself', '., 'weight', 'Ġ=', 'Ġtorch', '., 'randn', '(', 'input', '_,
'size', ',', 'Ġoutput', '_, 'size', ')', 'ĠĠĠĠĠĠĠĠ', 'Ġself', '., 'bias', 'Ġ=', 'Ġtorch', '., 'zeros', '(',
'output', '_, 'size', ')', 'ĠĠĠĠĠ', 'Ġdef', 'Ġ__', 'call', '__(', 'self', ',', 'Ġx', '):', 'ĠĠĠĠĠĠĠĠ',
'Ġreturn', 'Ġx', 'Ġ@', 'Ġself', '., 'weights', 'Ġ+', 'Ġself', '., 'bias', 'ĠĠĠĠĠ']

```

What does this example show?

The tokenizer now captures **Python structure much better**.

1. Learned indentation tokens

Indentation tokens were observed earlier.

2. Learned double indentation

Example:

```
ĠĠĠĠĠĠĠĠĠ
```

Meaning:

newline followed by many spaces, which is common in code.

3. Common Python words appear as single tokens

Examples:

- `class`
- `init`
- `call`
- `self`
- `return`

These appear as **meaningful tokens**.

4. Handles `_` and `.` well

So both **snake_case** and **object access** work properly.

Examples:

- `input_size`
- `self.weight`

5. Handles camelCase

Example:

`LinearLayer`

Tokenized as:

`["Linear", "Layer"]`

So the tokenizer can also understand **camel case structure**.

This is very useful.

Saving the Tokenizer

If we want to reuse the tokenizer later, we need to save it.

Just like models, tokenizers can be saved using `save_pretrained()`.

```
tokenizer.save_pretrained("code-search-net-tokenizer")
```

This will create a folder named:

code-search-net-tokenizer

Inside this folder will be all files required to reload the tokenizer.

In simple terms:

- The tokenizer is saved
- A folder is created
- Later it can be loaded from that folder

Sharing with Others

You can also upload the tokenizer to the **Hugging Face Hub**.

Then your colleagues or others can use it.

First you must log in to your Hugging Face account.

Logging in from a notebook

```
from huggingface_hub import notebook_login
```

```
notebook_login()
```

A widget will appear where you can enter your login credentials.

Logging in from terminal

If you are not using a notebook:

```
huggingface-cli login
```

This logs you in through the terminal.

Uploading to the Hub

After logging in:

```
tokenizer.push_to_hub("code-search-net-tokenizer")
```

This will create a new repository named:

code-search-net-tokenizer

inside your namespace.

All tokenizer files will be uploaded there.

Loading the Tokenizer Later

After uploading, it can be loaded from anywhere using `from_pretrained()`.

```
tokenizer = AutoTokenizer.from_pretrained("huggingface-course/code-search-net-tokenizer")
```

Replace "`huggingface-course`" with your own namespace.

So the tokenizer can be loaded:

- From a **local folder**
- Or from the **Hugging Face Hub**

Key Summary

After creating a new tokenizer:

- Save it locally using `save_pretrained()`
- Optionally upload it to the Hub
- Login is required before uploading
- Later it can be loaded easily using `from_pretrained()`